ADS205 Decision, Computation and Language

Lecture 2 – Deterministic and Nondeterministic Finita Automata

Dr Chunchuan Lyu and Dr Xinhong Chen



Xi'an Jiaotong-Liverpool University
西交利物浦大學

Recap

- Finite Automata
- Regular Operations

Today

- Deterministic Finite Automata
- Nondeterministic Finite Automata



Recap: Basic Concepts

- A **string** is a **finite** sequence of characters, e.g., 01001.
- Computation is that given an input string, we follow the finite automaton and see if the string is accepted or not;
- An **alphabet** is a **finite** set of symbols,
- A **language** is a set of strings, which could be **empty, finite, or infinite**;
- A language is regular if some finite automaton recognizes it.



Recap: Regular Operations

- Regular operations on languages are union ∪, concatenation ∘, and star $*$. (**ONLY THESE THREE!**)
  - union $A \cup B = \{w | w \in A \; or \; w \in B\}$
  - concatenation $A \circ B = \{wy | w \in A \; and \; y \in B\} = AB$
  - star $A^* = \{w_1 \ldots w_k | each \; w_i \in A \; for \; k \geq 0\}$, $(w_0 = \varepsilon)$.
- Languages can be represented as **regular expressions**.
- Languages can be **closed** under certain operations.



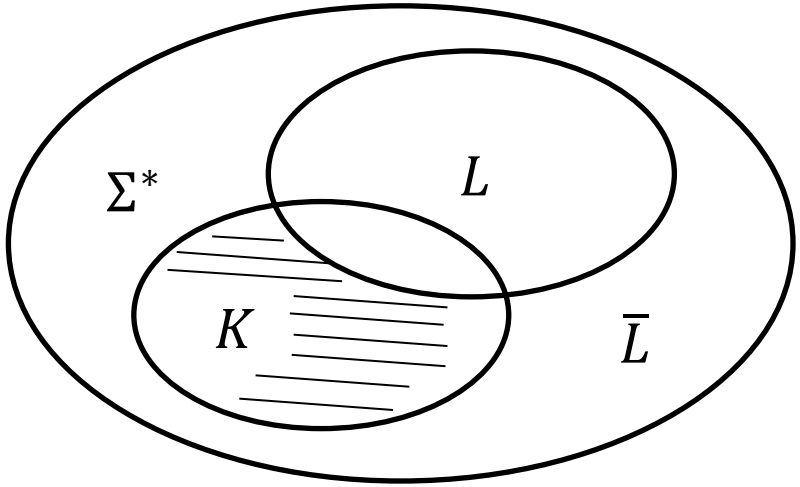
Recap: Star Operation

- star $A^* = \{w_1 \dots w_k | each\ w_i \in A\ for\ k \geq 0\}$, $(w_0 = \varepsilon)$.
- For each string in A^*, it is either ε $(k = 0)$ or it has the form of $w_1 \dots w_k$ $(k > 0)$, which is the repetition of the strings in A.
- Understand this process as we randomly take a string from A, we repeat this process k times, and we concatenate them together. These k strings can be different or the same.
- The string in AA^* is $ww_1 \dots w_k$, so we could understand this process as randomly take $k + 1$ times, so this is also in A^*.
- The number of all strings we could have for each k is $|A|^k$.



Recap: Complement Operation

- $\Sigma = \{a, b\}$
- Σ^* is the set of all strings formed by characters a and b.
- $L = \{w \in \Sigma^* | |w| \; is \; even. \}$
- $\bar{L} = \Sigma^* \backslash \mathrm{L}$
- $\bar{L} = \{w \in \Sigma^* | |w| \; is \; odd. \}$
- Assume we have a language K
- How do we compute $K \backslash L$?
- $K \backslash L = K \cap \bar{L}$





Recap: Tutorial Practice

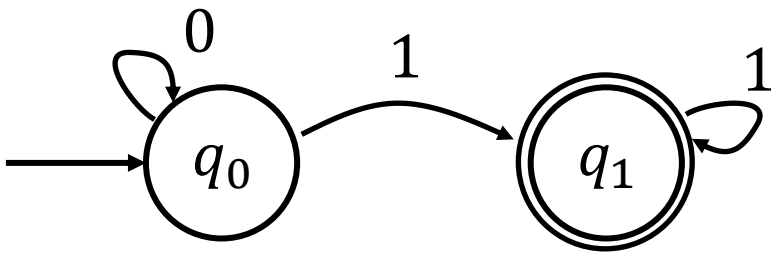
- $\Sigma = \{a, b\}$
- $FINITE$: all finite languages over Σ
- If $L \in FINITE$, then L is finite, containing limited number of strings, like $\{a, b, ab, aa, ba\}, \{a, b\}, \{aaa, bbb\}$
- $coFINITE = \{L \subseteq \Sigma^* | \bar{L} \in FINITE\}$, $\bar{L} = \Sigma^* \backslash \mathrm{L}$
- If $L \in coFINITE$, then its complement $\bar{L}$ is finite, and L is infinite, like $\{w \in \Sigma^* | |w| \geq 3\}, \Sigma^*, \Sigma^* \backslash \{\varepsilon\}$
- $FINITE \cap coFINITE = \emptyset$, $FINITE \cup coFINITE$ include all finite languages and the infinite languages whose complement are finite, but **not** the infinite languages whose complement are infinite.



Recap: Q&A

- What if we read a character but there is no corresponding transition?

- Example:



$\Sigma = \{0,1\}$

- For this given automata, it should **reject** the input string.

Recap: Finite Automata Definition

A finite automaton M is a 5-tuple vector $(Q, \Sigma, q_0, F, \delta)$:

- Q: a finite set of states
- Σ: a finite set of alphabet characters (i.e., all possible characters)
- $q_0 \in Q$: the start state
- $F \subseteq Q$: the set of accept/final states => *could be empty*
- $\delta: Q \times \Sigma \rightarrow Q$ transition functions

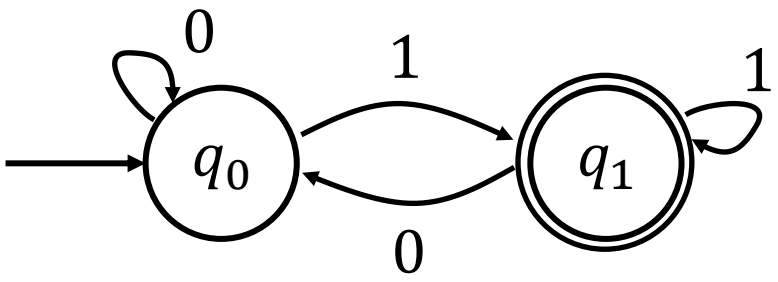


Deterministic Finite Automata (DFA)

	0	1
q_0	q_0	q_1
q_1	q_0	q_1

- **Key Characteristics:**
- **Deterministic**: For every state and every input symbol, there is exactly one defined next state.
- **Every state has an outgoing transition for all possible inputs.**

- Example:

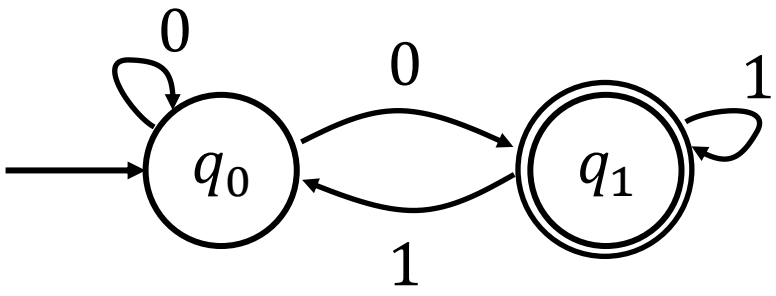


$\Sigma = \{0,1\}$

- For a given string w and a DFA M, the computation of w on M is **fixed**, or following the definition, **deterministic**.

Nondeterministic Finite Automata (NFA)

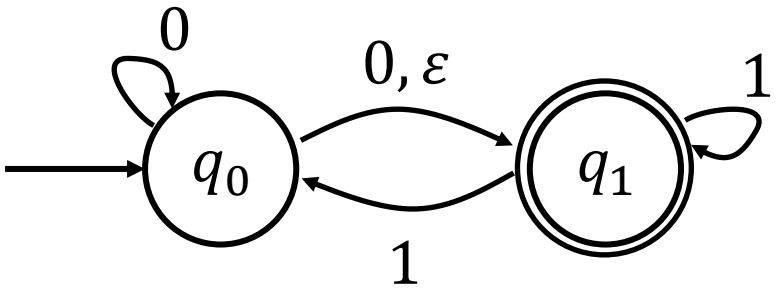
- **Loose the constraints**?
  - have no outgoing transition for a character in a state;
  - have multiple outgoing transitions for the same character;
- **Multiple possible paths** to conduct the computation
- **If some paths lead to the accept state** => the string is accepted.
- Example: 00, 11



$\Sigma = \{0,1\}$

Nondeterministic Finite Automata (NFA)

- Make it freer?
- ε-transitions: move the machine to the next state **without reading input characters**. (NOT SKIPPING INPUT!!!)
- Offers additional choices
- Example: 00, 11



$\Sigma = \{0,1\}$

Nondeterministic Finite Automata (NFA): Formal

An NFA M is a 5-tuple vector $(Q, \Sigma, q_0, F, \delta)$:

- Q: a finite set of states
- Σ: a finite set of alphabet characters (i.e., all possible characters)
- $q_0 \in Q$: the start state
- $F \subseteq Q$: the set of accept/final states
- δ: $\boldsymbol{Q} \times (\boldsymbol{\Sigma} \cup \{\boldsymbol{\varepsilon}\}) \rightarrow \boldsymbol{\mathcal{P}(Q)}$ transition functions

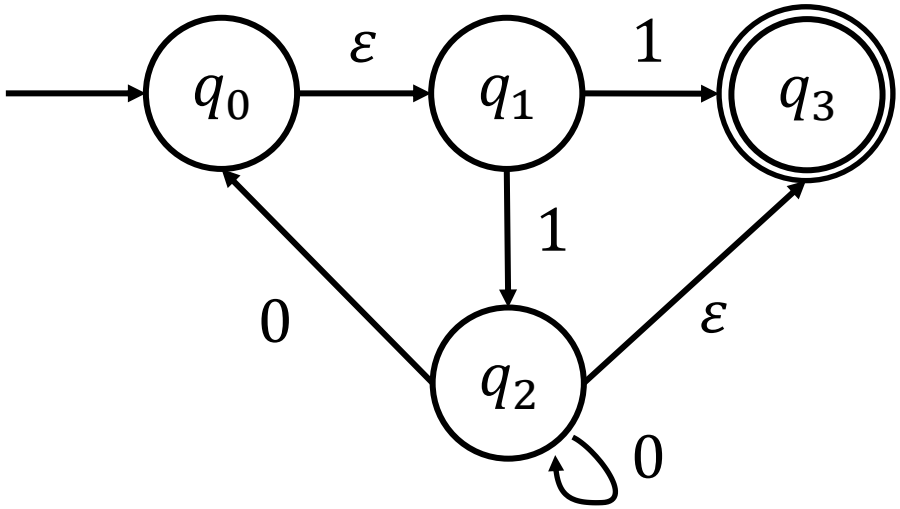
$\mathcal{P}(Q)$: the powerset of Q, i.e., the set of all subsets of Q

$\Sigma \cup \{\varepsilon\}$ can also be written as Σ_ε



Nondeterministic Finite Automata (NFA)

- Example



$\Sigma = \{0,1\}$

- $w = 10$, accept or not?
- $w = 101$, accept or not?
- $w = 01$, accept or not?
- $w = 10101$, accept or not?



Relations between DFAs and NFAs

- **Based on definitions, NFAs include DFAs**
- For DFAs, every state has an outgoing transition for all inputs, and the computation path is unique. => all fulfilling NFA's definition

➢ Every DFA is an NFA.

☐ Reversely, are NFAs DFAs?

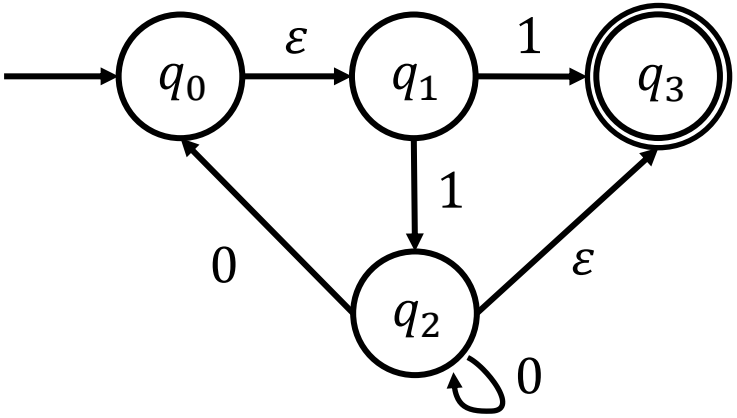
☐ In terms of definition, **NO**

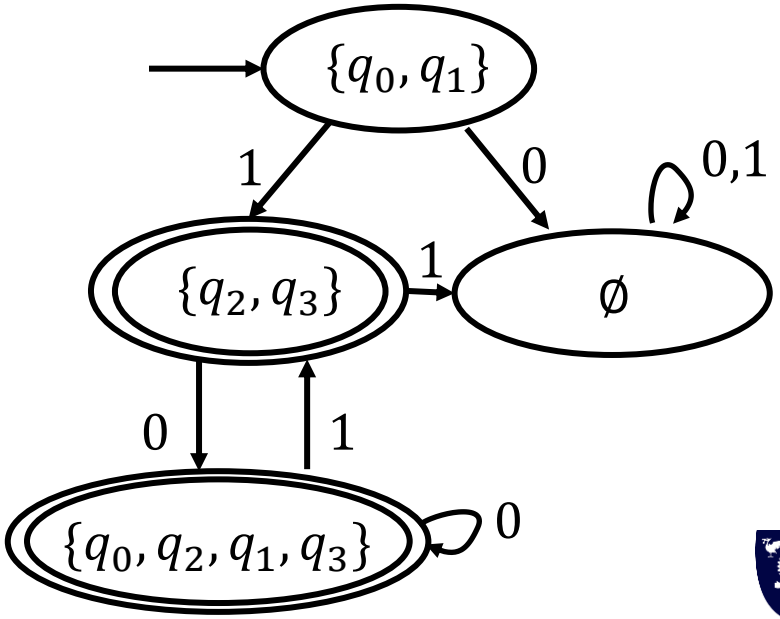
☐ But if we could **build a DFA for an NFA**, then they are equivalent.



Relations between DFAs and NFAs

- build a DFA for an NFA:
- Simulate the NFA, and for each transition, put all **reachable** NFA states in the DFA states. => **Powerset / Subset Construction**





Relations between DFAs and NFAs

Theorem: If an NFA recognizes a language A then A is regular.

Proof: Let an NFA $M = (Q, \Sigma, q_0, F, \delta)$ recognize A, construct a DFA $M' = (Q', \Sigma, q_0', F', \delta')$ recognizing A

- Construction of M':
- $Q' = \mathcal{P}(Q)$
- $q_0' = \varepsilon\text{-closure}(\{q_0\})$
- $F' = \{R \in Q' | R \cap F \neq \emptyset\}$
- $\delta' : Q' \times \Sigma \to Q'$,

$$for\ R \in Q', a \in \Sigma, \delta'(R, a) = \varepsilon\text{–closure}(\bigcup_{r \in R} \delta(r, a))$$

$$\varepsilon\text{–closure}(R) = \bigcup_{r \in R} \varepsilon\text{–closure}(r)$$

ε–closure$(r) = \{p \in Q \mid p$ is reachable from r through zero or more ε–transitions$\}$



Relations between DFAs and NFAs

Proof: Let an NFA $M = (Q, \Sigma, q_0, F, \delta)$ recognize A, construct a DFA $M' = (Q', \Sigma, q_0', F', \delta')$ recognizing A

- If M has n states, how many states could M' have at most by this construction?
- 2^n
- For each state, either it is in a subset or not, so 2 possible choices
- n states, so 2^n possible situations.



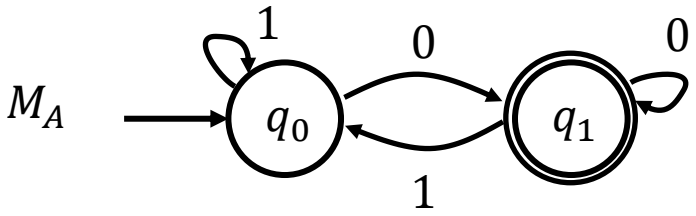
Recall: Regular Operations on Languages

- **Closure Properties for Regular Languages**
- Let C be a set of languages, and $\oplus$ denote a regular operation
- if applying $\oplus$ to languages in C results in a language in C, we say C is closed under $\oplus$.
- Proved example: if A and B are both regular languages, so is $A \cup B$.
- How about concatenation and other operations?
- Need more context! => see if we could solve them now

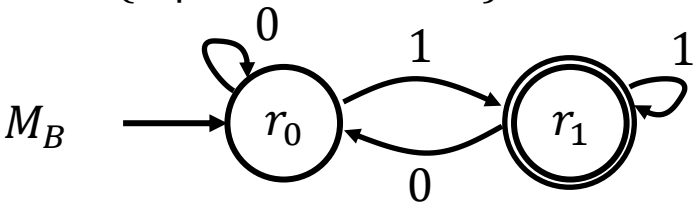


Closure of Regular Languages under Union

$A = \{w | w \text{ ends with } 0\}$

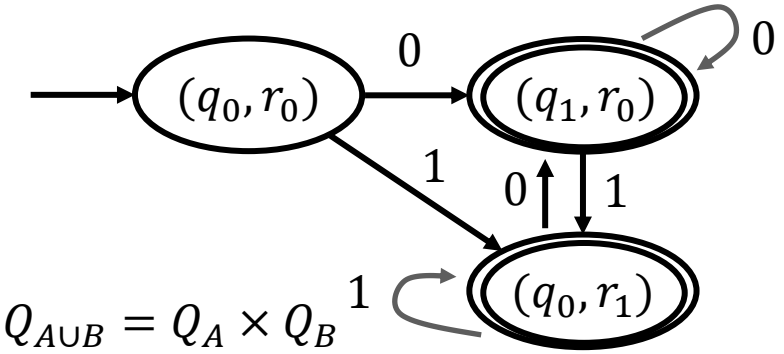


$B = \{w | w \text{ ends with } 1\}$



$A \cup B = \{w | w \text{ ends with } 0 \text{ or } 1\}$.

"simulate" both machines simultaneously



$Q_{A \cup B} = Q_A \times Q_B$

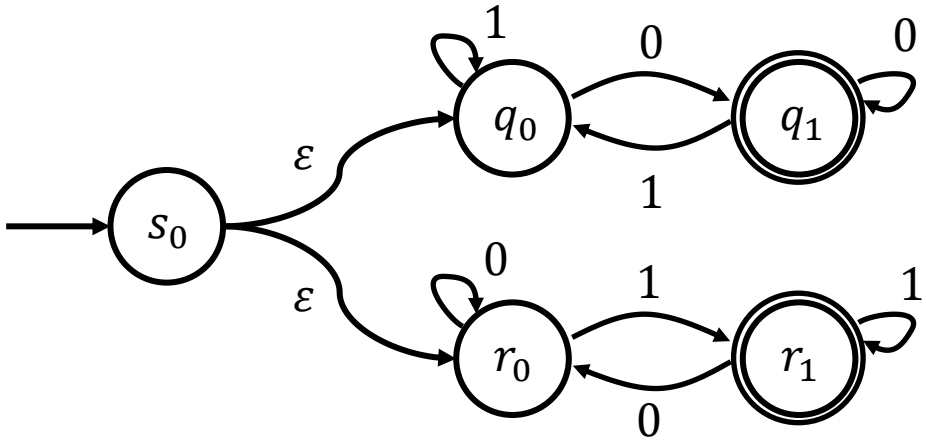
- **How to do this using NFAs?**
- Our goal: As long as w is accepted by M_A or M_B
  - As long as **there is a path** leading to the accept states in M_A or M_B.



Closure of Regular Languages under Union

- **How to do this using NFAs?**
- Our goal: As long as w is accepted by M_A or M_B
  - As long as **there is a path** leading to the accept states in M_A or M_B.
  - Offering a choice between M_A or M_B

$A = \{w | w \ ends \ with \ 0\}$

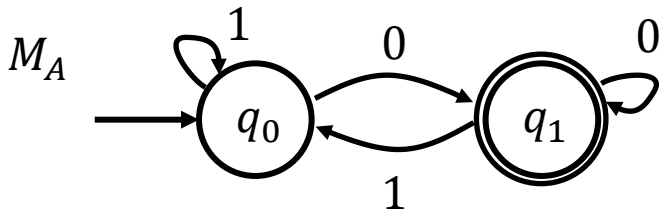


$B = \{w | w \ ends \ with \ 1\}$

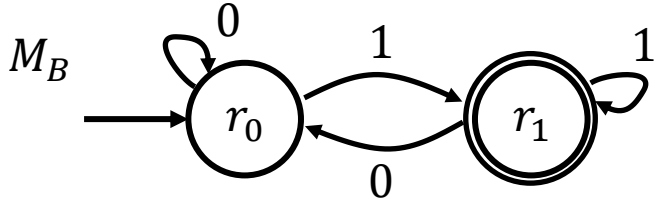
Closure of Regular Languages under Concatenation

- $A \circ B = \{wy | w \in A \; and \; y \in B\} = AB$
- Example:

$A = \{w | w \; ends \; with \; 0\}$



$B = \{w | w \; ends \; with \; 1\}$

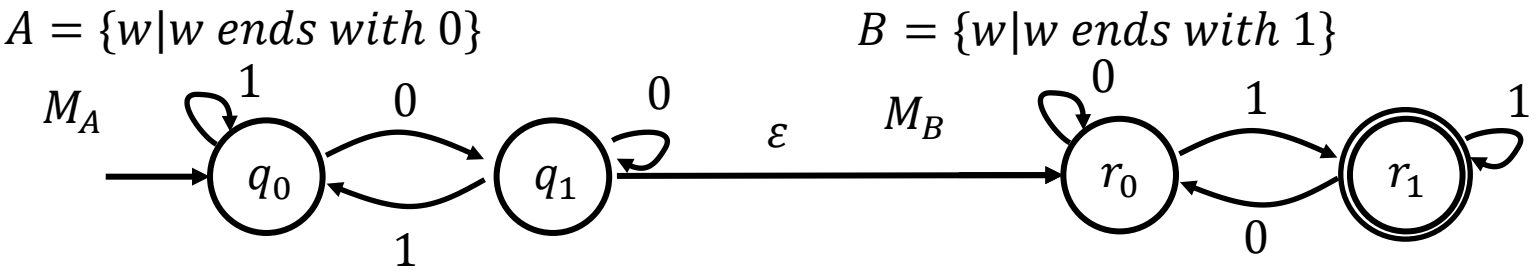


- Concatenate A with B => A first (**ORDER MATTERS!**)
- For $wy \in AB$, $w \in A$, w will end in the accept state of M_A
- Similar for y
- understand wy as going through M_A and accept first, and then go through M_B and accept



Closure of Regular Languages under Concatenation

- $A \circ B = \{wy | w \in A \; and \; y \in B\} = AB$
- Example:



- The key is to bridge M_A and M_B, **linking the accept state of M_A to the start state of M_B**
- use ε-transitions! But remember to **modify** the original accept states.

Theorem: if A and B are both regular languages, so is $A \circ B$.

Equivalently, ***regular languages are closed under concatenation*.**

Closure of Regular Languages under Star

- $A^* = \{w_1 \ldots w_k | each\ w_i \in A\ for\ k \geq 0\}, (w_0 = \varepsilon)$
- two types of strings are included in A^*:
  - the empty string ε
  - all the strings generated by product k
  - For each w, $w \in A$, so the state sequence should start with q_0 and end with q_{ac}

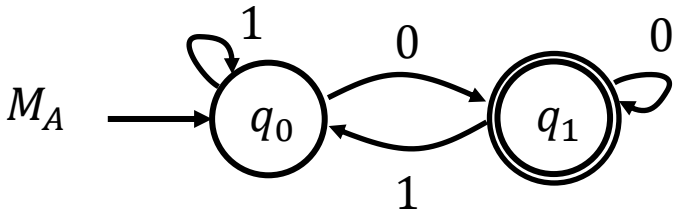
$$\overset{w_1}{q_0 \cdots q_{ac}} \rightarrow \overset{w_2}{q_0 \cdots q_{ac}} \rightarrow \overset{\cdots\cdots}{} \overset{w_k}{q_0 \cdots q_{ac}}$$

  - To concatenate between them => link q_{ac} to q_0

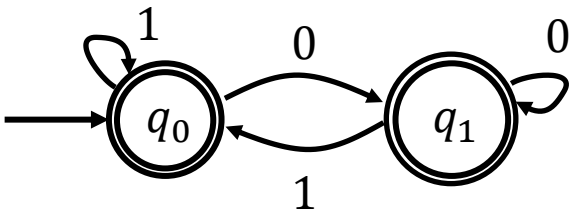


Closure of Regular Languages under Star

- $A^* = \{w_1 \ldots w_k | each\ w_i \in A\ for\ k \geq 0\}, (w_0 = \varepsilon)$
- Example: $A = \{w | w\ ends\ with\ 0\}$
- two types of strings are included in A^*:
  - the empty string ε
  - all the strings ending with 0, i.e., A itself



- **Key**: make the empty string ε have an accept state
- Trials:

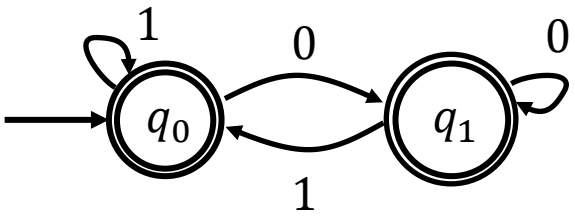


✓ the empty string ε have an accept state

× strings not ending with 0 also accept

Closure of Regular Languages under Star

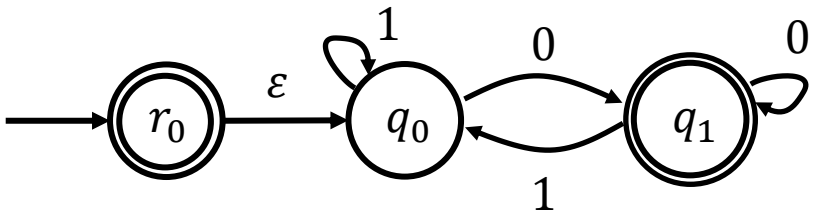
- Trials:



✓ the empty string ε have an accept state

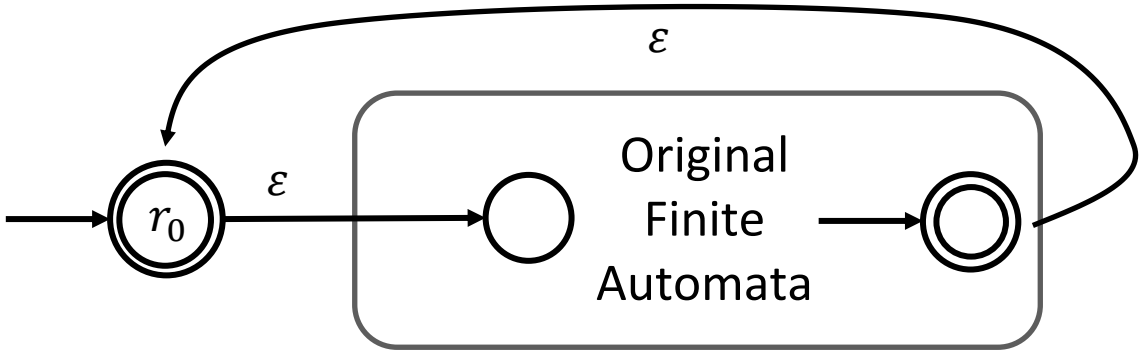
× strings not ending with 0 also accept

- Tip: Try **NOT** to change the original finite automata
- What we want: if the string is empty -> accept; if non-empty, follow M_A



Closure of Regular Languages under Star

- More generalized form:
- (1) create new start state, (2) make ε accept and, (3) loop back



Theorem: if A is a regular language, so is A^*.

Equivalently, ***regular languages are closed under star*.**

Quick Review

- In a Deterministic Finite Automata, **every** state has an outgoing transition for **all possible input characters**.
  - Note: For a given string w, the computation path over DFA is deterministic, i.e., **there is only one path**.
- In a Nondeterministic Finite Automata, there exist **any number** of **transitions** for each state, and any number of **computation paths**.
- For NFAs, systems will **automatically** **make choices** during computation and accept a string as long as there is one path leading to accept states.
- NFAs and DFAs are **equivalent**. (**NOT THE SAME!!!**)



Q & A

① Must a DFA have exactly one accepting state?

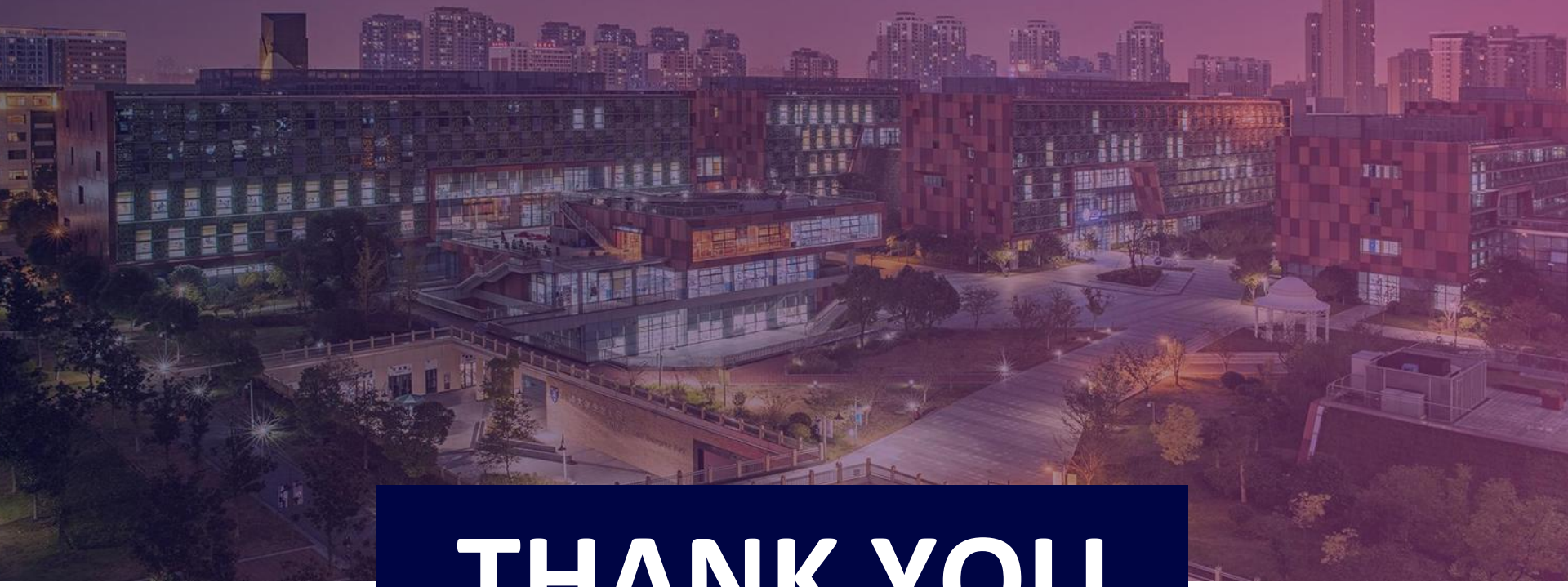
② In an NFA, if the current state has no transition on the next input symbol, what happens?

③ Can an NFA accept a string even if some of its computation paths reject that string?

④ Is it possible for a computation to stay in some states without the self-loop transition?

⑤ Can two different DFAs recognize the same language?





THANK YOU

Xi'an Jiaotong-Liverpool University
西交利物浦大學

XJTLU | SCHOOL OF FILM AND TV ARTS